Technical Challenge - Fullstack Developer

Task: Build a Kanban-Style To-Do App Using MERN Stack

Summary

Develop a task management app using the MERN stack: React.js with Redux, TypeScript, Node.js, MongoDB, and Mongoose. Implement React Server Components where suitable. Use Redux Toolkit for state management when possible. Employ AI tools to assist the development.

Application Requirements

Screens & Routing

The app should have two screens, each accessible via separate routes.

Main Board View (Kanban Layout)

Display tasks organized into three columns.

- To Do
- In Progress
- Done

Each task must display at least:

- Title (required)
- Due Date (required)
- Optional description
- Move tasks between columns via drag-n-drop and via a dropdown/context menu.
- Maintain task order within each column and update it when tasks move.

Task Creation View

- Provide a form to create new tasks.
- Validate form input with clear error messages for invalid entries (e.g., empty title).
- Save tasks to MongoDB via the backend API upon submission.

Functionality

Move Tasks

Each task should be draggable and movable to any column. While dragging, the task should visually float following the mouse or touch movement. Activate all available drop zones with visual feedback, highlighting the probable drop zone. When moving a task via a dropdown or

context menu, display the correct list of columns excluding the current one. For example, if the task is in **To Do**, the menu should list **In Progress**, and **Done**. Update status and order accordingly.

Data Persistence

Store all task data (title, description, status, order) in MongoDB using Mongoose models.

Redux Integration

Abstract all server interactions (fetching, creating, updating tasks) through Redux state management. Avoid direct API calls in components.

Feedback & UX

- Apply smooth animation during dragging and dropping.
- Show loading indicators when fetching or updating data.
- Display user-friendly notifications for actions like creation, movement, and errors.
- Ensure the UI is responsive and provides appropriate UX across screen sizes.
- Ensure readability with proper padding, spacing, and alignment—pixel-perfect design is not required.

Tech Stack

Frontend

- React.js
- Redux/Redux Toolkit
- TypeScript
- Tailwind CSS

Backend

- Node.js
- Express.js

Database

- MongoDB with Mongoose

Constraints

- Use any third-party libraries or UI frameworks to reduce boilerplate and development time.
- Do not use any in TypeScript.
- Upload the project to a Git repository and share the link.